

Jenkins Infrastructure & Server Setup Manual

A comprehensive guide for setting up an isolated, secure, and cost-effective Jenkins automation server on Microsoft Azure from scratch.

1. Introduction & Requirements

Jenkins is an open-source Automation Server used to implement Continuous Integration and Continuous Deployment (CI/CD) pipelines. It automates building, testing, and deploying applications so code changes can reach production safely and reliably.

This setup separates Infrastructure Provisioning (handled automatically via an Azure DevOps pipeline) from Software Installation (handled manually for complete control and debugging visibility).

Prerequisites & System Requirements

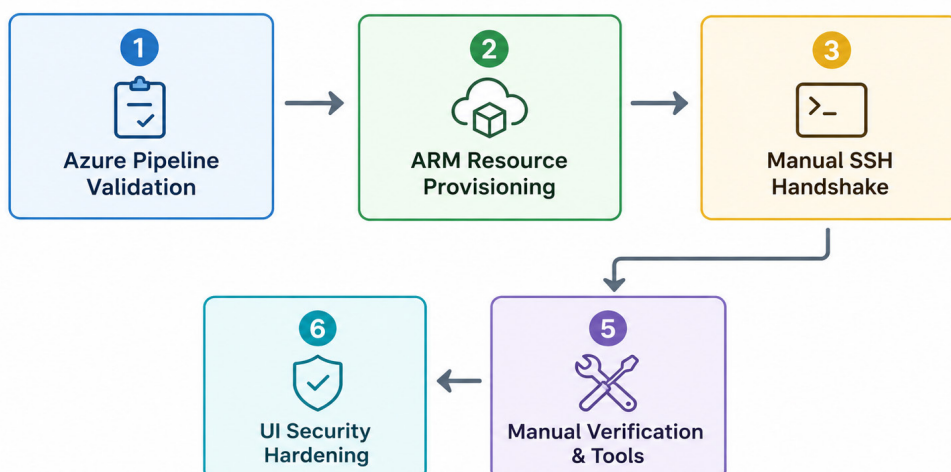
To build this setup, you need the following access and tools:

- **Azure Account:** An active subscription with permissions to create Resource Groups and Virtual Machines.
- **Azure DevOps Account:** Access to run a pipeline with a configured Service Connection (*azure-service-connection*).
- **SSH Key Pair:** An SSH public/private key pair (*ssh-rsa* format) to securely connect to the Linux machine without passwords.
- **VM Specifications** (Target):
 - **OS:** Ubuntu Server 22.04 LTS (Jammy Jellyfish).
 - **Size:** *Standard_B2s* (2 vCPUs, 4 GB RAM) or *Standard_D2s_v3*.
 - **Disk:** 64 GB Premium SSD storage.

2. Infrastructure Architecture & Flow

Deployment Flow

The lifecycle of this setup follows a strictly organized progression:



← Jobs in run #20260610...

jenkins-infra

✓ [1/2] Validate

> ✓ ARM validate + what-if 7m 5s

✓ [2/2] Deploy

✓ Deploy Infrastructur... 1m 29s

Initialize job 3s

Checkout jenkins-infra... 1s

Provision Azure R... 1m 17s

Output Connection D... 6s

Post-job: Checkout j... <1s

Finalize Job <1s

Finalize build

Report build status <1s

✓ Output Connection Details

View raw log

```

22 (Use 'node --trace-deprecation ...' to show where the warning was created)
23 /usr/bin/az login --service-principal -u *** --tenant 4daf1e3c-f832-41c7-91d0-0584ae8e64c3 --allow-no-subscriptio
24 [
25   {
26     "cloudName": "AzureCloud",
27     "homeTenantId": "4daf1e3c-f832-41c7-91d0-0584ae8e64c3",
28     "id": "43516bc1-a03a-4242-a816-62f472b818d0",
29     "isDefault": true,
30     "managedByTenants": [],
31     "name": "Azure subscription 1",
32     "state": "Enabled",
33     "tenantId": "4daf1e3c-f832-41c7-91d0-0584ae8e64c3",
34     "user": {
35       "name": "***",
36       "type": "servicePrincipal"
37     }
38   }
39 ]
40 /usr/bin/az account set --subscription 43516bc1-a03a-4242-a816-62f472b818d0
41 /usr/bin/bash /home/vsts/work/_temp/azureclitaskscript1781126075069.sh
42
43
44 Jenkins VM - Deployment Summary
45
46 Resource Group : rg-jenkins-infra-dev
47 VM Name       : jenkins-vm
48 Public IP    : 20.109.112.147
49 Next Step Command:
50 ssh azureuser@20.109.112.147
51
52
53 /usr/bin/az account clear
54 Finishing: Output Connection Details

```

3. Infrastructure Files (Code Contents)

To provision the network and virtual machine, the infrastructure repository uses three core files located inside the code repository.

A. Core ARM Template (*azuredeploy.json*)

This file defines the literal Azure resources to construct. It sets up an isolated Virtual Network (VNet) with a subnet, maps a Network Security Group (NSG) opening ports 22 (SSH), 8080 (Jenkins UI), and 50000 (Jenkins Agent), requests a Static Public IP address, and mounts an Ubuntu 22.04 LTS Generation 2 virtual machine.

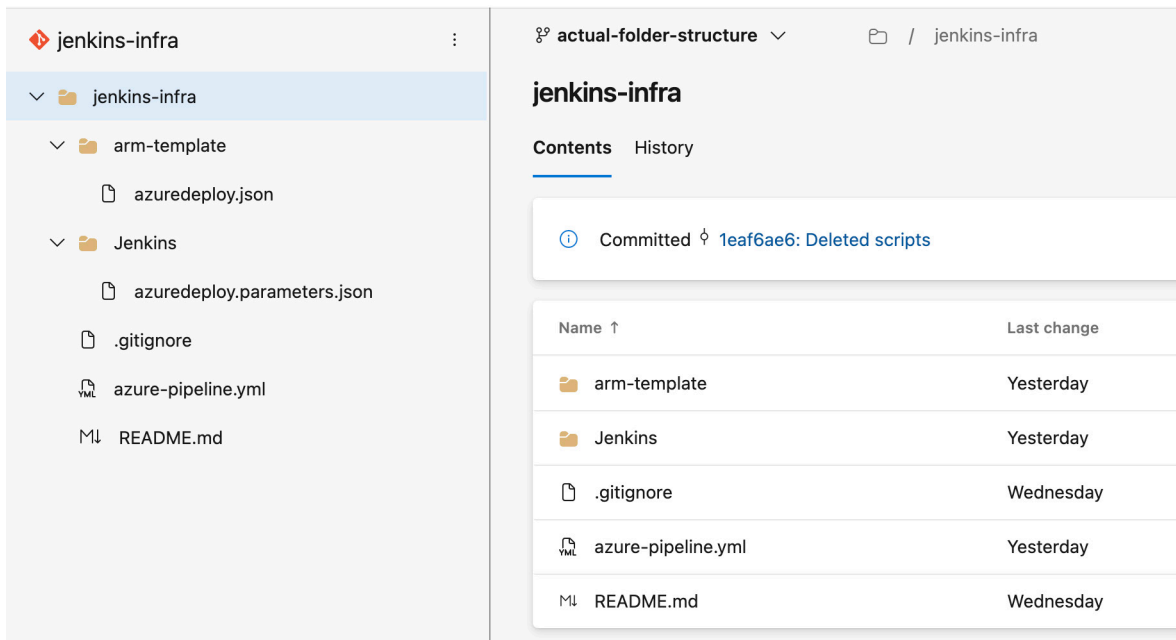
B. Input Parameters (*azuredeploy.parameters.json*)

This companion file supplies custom input settings directly to your infrastructure blueprint. It specifies parameters like your infrastructure environment staging tag (*dev*), standard VM size configurations (*Standard_B2s*), deployment region location targets (*eastus2*), and binds your unique public SSH security signature key.

C. Azure DevOps Pipeline Blueprint (*azure-pipelines.yml*)

The automated orchestrator file split into two operational blocks:

- **Stage 1 (Validate):** Verifies the ARM format integrity, creates your resource group (*rg-jenkins-infra-dev*) if it is missing, and previews your infrastructure modifications.
- **Stage 2 (Deploy):** Compiles the variables and commands to run the production deployment, rendering your live VM public IP configuration metrics out to the console logs.



refer - <https://github.com/kkinfrasptvlttd/jenkins-infra/tree/actual-folder-structure/jenkins-infra>

4. Manual Server Configuration Step-by-Step

Once your Azure DevOps pipeline completes successfully, copy the output Public IP from your logs and execute these terminal commands sequentially.

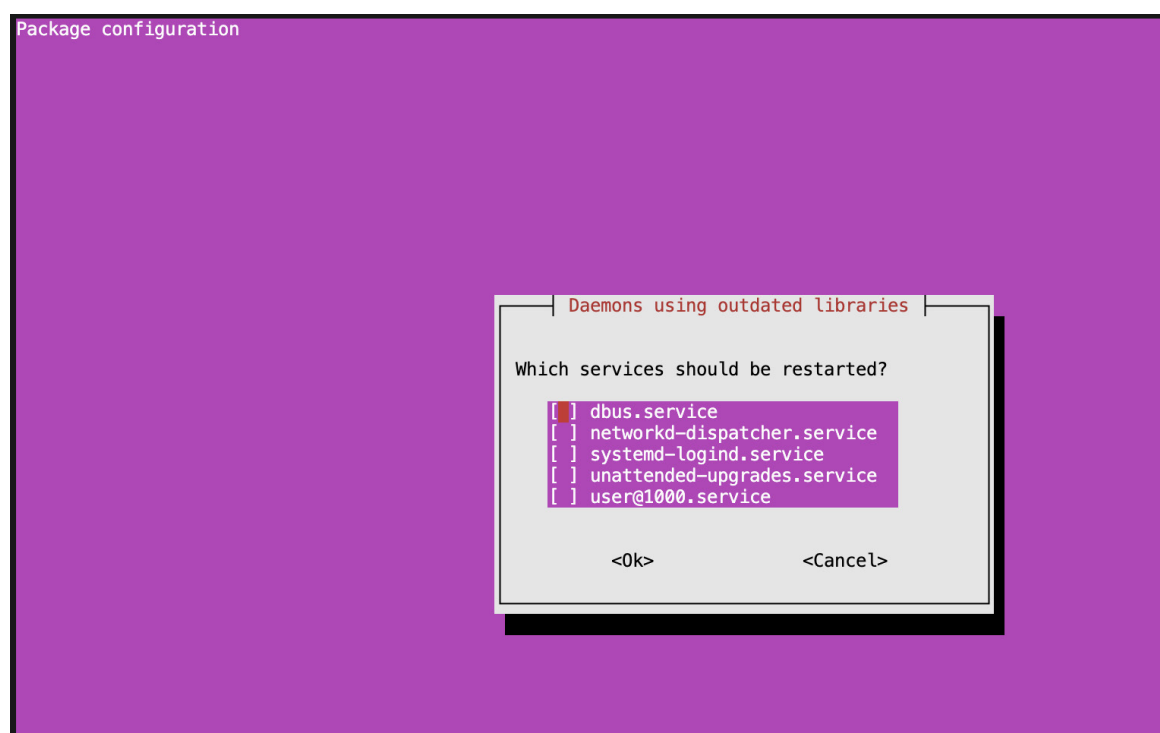
Step 1: Establish Connection and Update Package Indexes

Open your local computer terminal and open an SSH shell session to enter the VM:

```
ssh azureuser@<YOUR_VM_PUBLIC_IP>
```

Once inside the Ubuntu command line, pull down index logs for package definitions and patch security items:

```
sudo apt-get update -y && sudo apt-get upgrade -y
```



Step 2: Install Java Development Kit 21

Jenkins requires Java to run. Modern versions of Jenkins require Java 21 or Java 25:

```
sudo apt-get install -y openjdk-21-jdk
```

Verify the active configuration:

```
java -version
```

Step 3: Register the Stable Jenkins Repositories

Import the modern cryptographically signed public verification key and register the Debian stable download repository route:

```
# Clean out stale legacy references if any exist  
sudo rm -f /etc/apt/sources.list.d/jenkins.list /usr/share/keyrings/jenkins-keyring.asc
```

```
# Import up-to-date signing key entries  
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key | sudo tee  
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
# Re-add repository definitions securely  
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]  
https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list  
> /dev/null
```

Refresh your local system indexes and deploy the core service application engines:

```
sudo apt-get update -y  
sudo apt-get install -y jenkins
```

Instruct the operating system to start Jenkins and preserve its execution whenever the underlying server reboots:

```
sudo systemctl enable jenkins  
sudo systemctl start jenkins
```

Step 4: Add Build Engine Prerequisites (Git, Maven, and Docker)

Install the essential DevOps dependencies for cloning code, compiling packages, and building containers:

```
# Deploy core utility items  
sudo apt-get install -y git maven apt-transport-https ca-certificates curl  
software-properties-common
```

```
# Set up the secure Docker package repository keys
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Install the Docker runtime engine
sudo apt-get update -y && sudo apt-get install -y docker-ce docker-ce-cli containerd.io
sudo systemctl enable docker && sudo systemctl start docker
```

Step 5: Configure Permissions and Install Command Line Interfaces (CLIs)

Add both the standard user and Jenkins execution processes to the Docker security group so they can run containers without using *sudo*:

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker azureuser
```

Deploy the Azure CLI and the Node.js runtime environments:

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - && sudo apt-get install
-y nodejs
```

Step 6: Configure the Local Firewall (UFW)

Enforce access restrictions on the server's internal firewall:

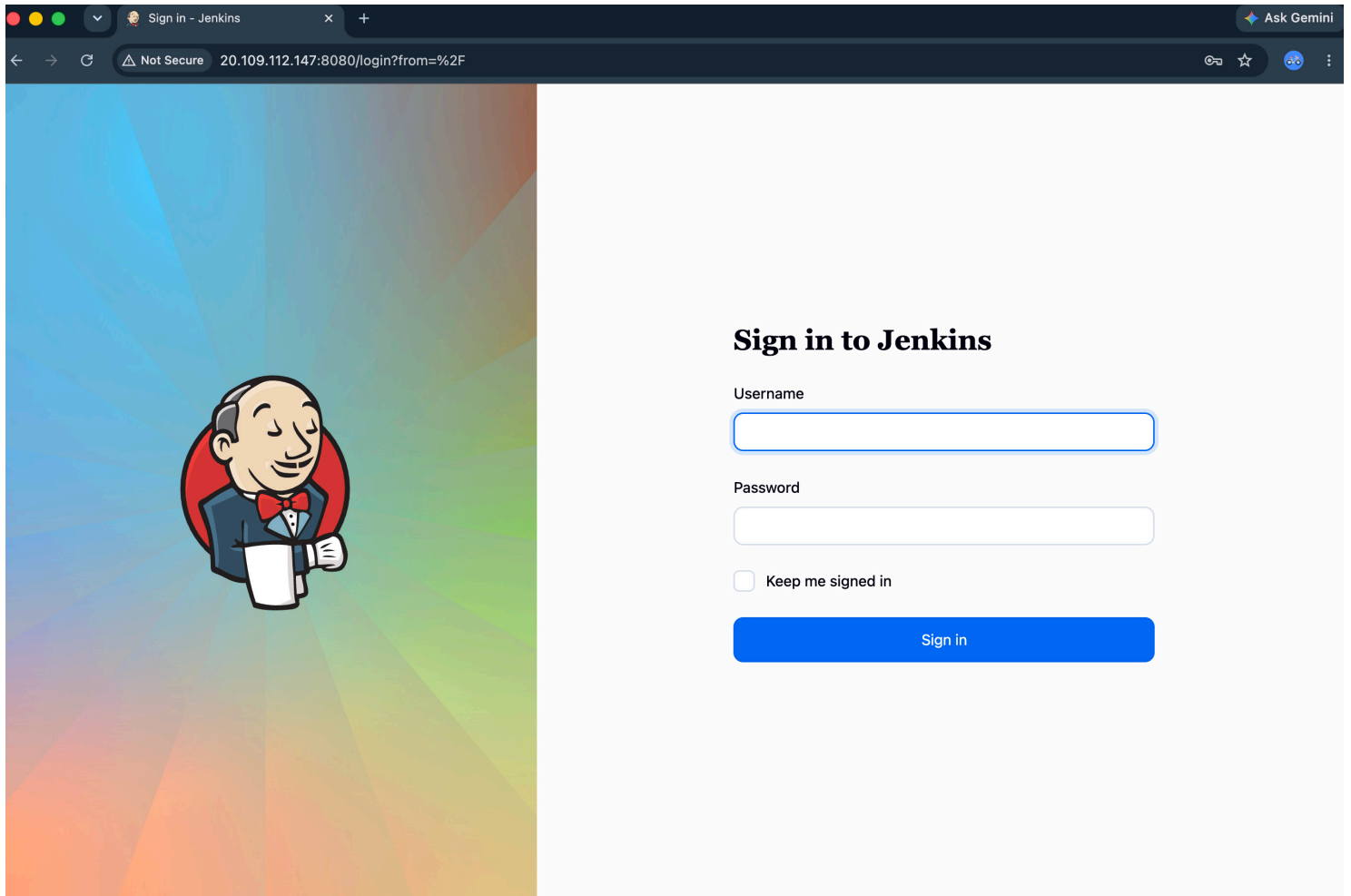
```
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 8080/tcp comment 'Jenkins UI'
sudo ufw allow 50000/tcp comment 'Jenkins Agent'
sudo ufw --force enable
```

Step 7: Retrieve Unlock Credentials

Print out the temporary initialization password created during setup:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy that string, open a web browser to http://<YOUR_VM_PUBLIC_IP>:8080, paste the password, select **Install Suggested Plugins**, and create your customized primary master administrator account credentials.



5. Security & User Management Hardening

Securing the Access Matrix Matrix

By default, the initial setup can leave permissions wide open. You must restrict who can modify your server.

- Navigate to **Manage Jenkins → Security**.
- Under Authorization, select **Matrix-based security**.
- Locate the **Anonymous** user line and remove all checked options. This blocks unauthorized internet users from viewing your jobs.
- Click **Add user...**, enter your administrative login username, and grant yourself Administer rights (checking this column auto-fills all other columns in that row).
- Click **Add user...** for your teammates and grant them restricted developer roles (e.g., check *Overall: Read*, *Job: Build/Cancel/Read*, and *View: Read*). Click Save.

6. Business Values & Efficiency

Use Cases

- **Automated CI/CD Workloads:** Clones source code repositories on every commit to test and build software artifacts.
- **Infrastructure as Code Validation:** Validates ARM templates or Terraform changes before deploying them to cloud platforms.
- **Containerization Pipelines:** Builds Docker images, uploads them to registries like Azure Container Registry (ACR), and runs container tests.

Strategy Advantages (Pros)

- **High Controllability:** Manual installations let you control exact patch levels and run direct diagnostic tracking.
- **Secure Isolation:** Network security blocks unwanted ports, while internal firewalls protect the host.
- **Low Idle Costs:** The server stops completely when not in use, keeping cloud storage costs minimal.

Cost Optimization (Auto-Shutdown Filter)

To prevent your Azure budget from consuming unnecessary cash when developers are offline, configure automatic deallocation:

1. Go to the Azure Portal, locate your **Virtual Machine**, and select **Auto-shutdown**.
2. Flip the state selector flag to **On**.
3. Input **18:00:00** into the target scheduled time entry box and select **(UTC+05:30) India Standard Time (IST)**.
4. Enable notification emails so you can skip or delay the shutdown if your team needs to work late.

The screenshot displays the Microsoft Azure portal interface. At the top, there's a navigation bar with 'Microsoft Azure', an 'Upgrade' button, and a search bar. Below this, the breadcrumb trail shows 'Home > Compute infrastructure | Virtual machines > jenkins-vm'. The main heading is 'jenkins-vm | Auto-shutdown', with a clock icon and a 'Virtual machine' subtitle. A search bar and action buttons (Save, Discard, Feedback) are present. On the left, a sidebar lists navigation options: Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Monitor, Resource visualizer, Connect, Networking, Settings, Availability + scale, Security, Backup + disaster recovery, and Operations. The 'Auto-shutdown' settings are shown on the right, with a table-like structure:

Enabled	☒ On ☐ Off
Scheduled shutdown	18:00:00
Time zone	(UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
Send notification before auto-shutdown?	☒ Yes ☐ No
Webhook URL ⓘ	
Email address ⓘ	kkinfrasptltd@outlook.com

At the bottom left, there's a tab labeled 'Auto-shutdown' with a clock icon.

7. Troubleshooting & Failures Faced

Throughout this setup, several technical blockers commonly occur. Here is how to diagnose and resolve them:

Problem 1: GPG Signature Validation Failure (*NO_PUBKEY*)

- **Symptom:** Running *apt-get update* errors out with *The following signatures couldn't be verified because the public key is not available.*
- **Root Cause:** The signing key repository location path changed or the historical authentication key file (like *jenkins.io-2023.key*) expired.
- **Resolution:** Clean out the old files using *rm*, download the updated key version (e.g., *jenkins.io-2026.key*), and update the repository sources index list.

Problem 2: Jenkins Engine Refuses to Boot (*status=1/FAILURE*)

- **Symptom:** Running *systemctl start jenkins* throws an error, and running *journalctl* shows: *Running with Java 17... which is older than the minimum required version (Java 21).*
- **Root Cause:** Modern LTS versions of Jenkins have dropped support for Java 8, 11, and 17. They now require Java 21 or 25.
- **Resolution:** Install the correct version using *sudo apt-get install -y openjdk-21-jdk*, then run *sudo update-alternatives --config java* to set it as the system default.

Problem 3: Docker Commands Throw "Permission Denied" Errors

- **Symptom:** Jenkins build logs fail during docker stages with *got permission denied while trying to connect to the Docker daemon socket.*
- **Root Cause:** The *jenkins* system service user account hasn't been added to the local host's Docker permission group.
- **Resolution:** Run *sudo usermod -aG docker jenkins* on your terminal, then reload the background process using *sudo systemctl restart jenkins*.

8. Future Scale Implementations

As your organization's automation needs expand, look into these advanced upgrades for your infrastructure:

- **Distributed Agent Architectures:** Instead of running resource-heavy builds directly on your master controller VM, configure Jenkins to dynamically spin up short-lived Azure Container Instances (ACI) or secondary agent machines to handle the workload.
- **Single Sign-On (SSO) Integration:** Migrate from manual user accounts to identity provider integrations. Use the Azure AD plugin to allow your team to log in using their official company Microsoft Entra ID credentials and enforce Multi-Factor Authentication (MFA).
- **Automated Backups:** Set up a cron task to periodically snapshot your */var/lib/jenkins/* configuration directories and store them safely inside an encrypted Azure Storage Account blob container for disaster recovery.